

Practice 4 – Half-Term Challenge: Grabbing an Object

Robotics and Cyber-Physical Systems
School of Sciences and Engineering
Tecnológico de Monterrey

Introduction

In this challenge, you will combine perception, coordinate transforms, and robot control to locate and pick up a red object. We will begin by exploring how an object detected by a camera is located relative to the robot.

Before You Begin

Open a terminal in the repository and update your copy:

```
1 git pull
2 uv sync
```

Make sure a red object is visible before starting the exercise.

Part 1 – Explore Object Localization

Objective

Run the existing red-object plotter to build an intuition for camera transforms and object localization:

```
1 uv run examples/plot_red_object.py
```

The camera window marks the detected object and displays its estimated coordinates in metres. The 3D window plots the camera and object relative to the robot base, where $+x$ is forward, $+y$ is left, and $+z$ is upward.

- Pan and tilt the head. The plotted camera should move while the object's base-frame position remains approximately stable.
- Move the robot or the red object and observe how the reported coordinates change.
- Press H to switch between head and wrist controls. Use I, J, K, and L to move them.

To compare both cameras, change the selected camera and run the script again:

```
1 CAMERA = WRIST_CAMERA
```

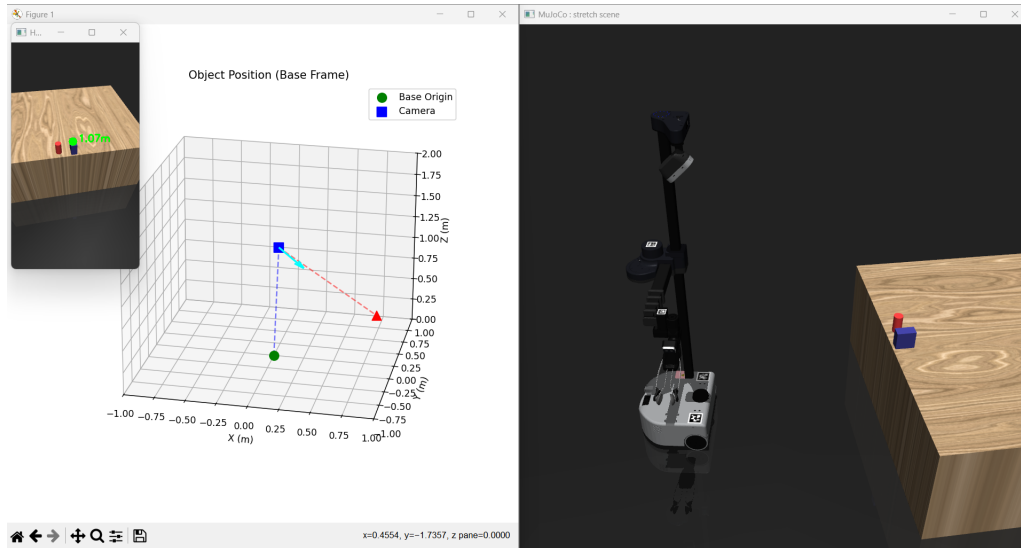


Figure 1: Camera detection and 3D visualization of an object's position relative to the robot.

Note

Small variations are expected from image detection and depth noise. When only the camera moves, the estimated object should still remain approximately fixed relative to the robot base.

Part A – Navigate Toward the Object

In this part, you will build `p4a_navigate.py` one behavior at a time. Run the program after adding each command function before continuing to the next one.

1. Copy the Boilerplate

Create the new script from the teleoperation boilerplate:

```
1 cp examples/teleop_boilerplate.py p4a_navigate.py
```

2. Add the Setup

Replace the imports at the top of the new script with:

```
1 import math
2
3 import cv2
4 import numpy as np
5
6 from stretch_tools import (
7     HEAD_CAMERA,
8     NormVelController,
9     RobotTransforms,
10    StateController,
11    TeleopProvider,
12    close_cameras,
13 )
14
15 try:
16     import stretch_body.robot as robot
17 except ImportError:
18     import stretch_mujoco_api.robot as robot
```

Add the controller constants below the imports:

```
1 KP_HEAD = 0.5
2 KP_ROTATION = 1.0
3 KP_DISTANCE = 5.0
4 TARGET_DISTANCE = 0.6
5 NAVIGATION_MAX_DISTANCE = 3.0
6 FLANK_ANGLE = -math.pi / 2
7 FLANK_TOLERANCE = math.radians(5)
8 LIFT_HEIGHT_OFFSET = 0.0
9 SCAN_SPEED = -0.2
```

Copy these perception functions above `main()`:

```
1 def detect_object(frame, depth_frame=None,
2                   max_distance=None, camera_info=None):
3     hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
4     mask = cv2.inRange(
5         hsv, np.array([0, 100, 100]),
6         np.array([10, 255, 255])
7     )
8     mask |= cv2.inRange(
9         hsv, np.array([170, 100, 100]),
10        np.array([179, 255, 255])
11    )
```

```

12     contours, _ = cv2.findContours(
13         mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
14     )
15
16     for contour in sorted(contours,
17                             key=cv2.contourArea,
18                             reverse=True):
19         if cv2.contourArea(contour) <= 100:
20             break
21         moments = cv2.moments(contour)
22         if not moments["m00"]:
23             continue
24
25         center = (
26             int(moments["m10"] / moments["m00"]),
27             int(moments["m01"] / moments["m00"]),
28         )
29         if depth_frame is not None and max_distance is not None:
30             distance = camera_info.get_depth(center, depth_frame)
31             if distance is None or distance > max_distance:
32                 continue
33         return center
34
35     return None
36
37
38 def locate_object(center, depth_frame, camera_info, camera_T):
39     depth = camera_info.get_depth(center, depth_frame)
40     if depth is None:
41         return None
42
43     x, y = camera_info.pixel_to_normalized(center)
44     object_T = np.eye(4)
45     object_T[:3, 3] = [x * depth, y * depth, depth]
46     return camera_T @ object_T

```

Inside `main()`, replace the controller setup after robot startup with:

```

1 controller = NormVelController(stretch, safe_base_mode=True)
2 teleop = TeleopProvider(robot=stretch)
3 transforms = RobotTransforms(stretch)
4
5 arm_stow = StateController(
6     stretch,
7     {
8         "arm_out": 0.0,
9         "wrist_pitch_up": -0.625,
10        "wrist_roll_counterclockwise": 0.0,
11        "wrist_yaw_counterclockwise": 2.994,
12        "gripper_open": 1.0,
13    },
14 )
15 lift_controller = StateController(stretch, {"lift_up": 0.3})
16 scanning_position = StateController(
17     stretch,
18     {
19         "head_pan_counterclockwise": 0.0,
20         "head_tilt_up": math.radians(-30),
21     },

```

```

22 )
23
24 command = None
25 head_rgb, head_depth = None, None
26 head_center = None
27 object_T = None
28 within_distance = False
29 stowed = False

```

The variables are declared before the loop so the nested command functions can use the latest camera data and command dictionary.

Use this camera and control structure inside the loop:

```

1  command = {}
2  command.update(arm_stow.get_command())
3
4  success, head_rgb, head_depth = HEAD_CAMERA.get_frames()
5  if not success:
6      return
7
8  head_center = detect_object(
9      head_rgb, head_depth, NAVIGATION_MAX_DISTANCE, HEAD_CAMERA
10 )
11 stowed = arm_stow.is_at_goal()
12
13 if head_center is not None:
14     cv2.circle(head_rgb, head_center, 5, (0, 255, 0), -1)
15
16 cv2.imshow("Head RGB", head_rgb)
17
18 command.update(lift_controller.get_command())
19 command = teleop.get_manual_override(command)
20 controller.set_command(command)
21
22 if cv2.waitKey(1) & 0xFF == ord("q"):
23     break

```

Finally, add camera cleanup to the finally block:

```

1  close_cameras()
2  cv2.destroyAllWindows()
3  stretch.stop()

```

Run the setup once before adding autonomous movement:

```

1  uv run p4a_navigate.py

```

3. Add Scanning

Add this nested function before the try block in main():

```

1  def scan():
2      command.update(scanning_position.get_command())
3      command["base_counterclockwise"] = SCAN_SPEED

```

Replace the simple detection check inside the loop with:

```

1  if head_center is None:
2      if stowed:
3          scan()

```

```

4 else:
5     cv2.circle(head_rgb, head_center, 5, (0, 255, 0), -1)

```

Run the script. The robot should first reach its safe stowed pose, then rotate while holding the head in the scanning position. Stop the program before continuing.

4. Add Head Tracking

Add the second nested function after `scan()`:

```

1 def track_with_head():
2     x, y = head_center
3     height, width = head_rgb.shape[:2]
4     horizontal_error = (x - width / 2) / (width / 2)
5     vertical_error = (y - height / 2) / (height / 2)
6     command["head_pan_counterclockwise"] = \
7         -KP_HEAD * horizontal_error
8     command["head_tilt_up"] = -KP_HEAD * vertical_error

```

Call it when the object is visible:

```

1 if head_center is None:
2     if stowed:
3         scan()
4 else:
5     track_with_head()
6     cv2.circle(head_rgb, head_center, 5, (0, 255, 0), -1)

```

Run the script again. When the red object enters the image, the base should stop scanning and the head should keep the object centered.

5. Add Navigation

Add the final nested command function:

```

1 def navigate_to_object():
2     nonlocal within_distance
3
4     object_x, object_y, object_z = object_T[:3, 3]
5     distance = math.hypot(object_x, object_y)
6     facing_error = math.atan2(object_y, object_x)
7
8     if within_distance:
9         within_distance = 0.5 <= distance <= 0.7
10    else:
11        within_distance = 0.55 <= distance <= 0.65
12
13    if within_distance:
14        flank_error = math.atan2(
15            math.sin(facing_error - FLANK_ANGLE),
16            math.cos(facing_error - FLANK_ANGLE),
17        )
18        command["base_counterclockwise"] = \
19            KP_ROTATION * flank_error
20
21        if abs(flank_error) <= FLANK_TOLERANCE:
22            lift_controller.desired_state["lift_up"] = \
23                object_z - LIFT_HEIGHT_OFFSET
24    else:

```

```

25     alignment = max(
26         0.0,
27         1.0 - abs(facing_error) / (math.pi / 2),
28     )
29     command["base_counterclockwise"] = \
30         KP_ROTATION * facing_error
31     command["base_forward"] = alignment * KP_DISTANCE * (
32         distance - TARGET_DISTANCE
33     )

```

After `track_with_head()`, locate the object and call the navigation function:

```

1  else:
2      track_with_head()
3      object_T = locate_object(
4          head_center,
5          head_depth,
6          HEAD_CAMERA,
7          transforms.get_cam_T(HEAD_CAMERA),
8      )
9      if object_T is not None and stowed:
10         navigate_to_object()
11         cv2.circle(head_rgb, head_center, 5, (0, 255, 0), -1)

```

Run the completed program. The robot should scan until it sees the red object, track it with the head, face and approach it, then rotate into a flank pose at approximately 0.6m. Forward movement is reduced while the robot is poorly aligned, and distance hysteresis prevents rapid switching near the target.

Part B – Grab the Object

Part A leaves the robot beside the object with the lift at the correct height. You will now switch to the wrist camera, align the gripper, extend the arm, close the gripper, and raise the object. Continue testing after every new function.

1. Copy Part A

Keep the completed navigation script and create Part B from it:

```
1 cp p4a_navigate.py p4b_grab.py
```

Add `time` and `WRIST_CAMERA` to the imports:

```
1 import math
2 import time
3
4 import cv2
5 import numpy as np
6
7 from stretch_tools import (
8     HEAD_CAMERA,
9     WRIST_CAMERA,
10    NormVelController,
11    RobotTransforms,
12    StateController,
13    TeleopProvider,
14    close_cameras,
15 )
```

Add the wrist and gripper constants:

```
1 KP_WRIST = 0.5
2 KP_ARM = 5.0
3 TARGET_WRIST_DISTANCE = 0.15
4 WRIST_CENTER_TOLERANCE = 0.75
5 ARM_DISTANCE_TOLERANCE = 0.025
6 GRAB_POSITION_SETTLE_TIME = 0.5
7 LIFT_TOP = 1.1
8 WRIST_X_OFFSET = 25
9 WRIST_Y_OFFSET = 75
```

2. Add the Grab Setup

After `scanning_position`, create the wrist and gripper controllers:

```
1 deploy_wrist = StateController(
2     stretch,
3     {
4         "wrist_pitch_up": 0.0,
5         "wrist_roll_counterclockwise": 0.0,
6         "wrist_yaw_counterclockwise": 0.0,
7     },
8 )
9 gripper_open_controller = StateController(
10    stretch, {"gripper_open": 130.0}
11 )
12 gripper_close_controller = StateController(
13    stretch, {"gripper_open": -25.0}
```

14)

Add the wrist data and grab state below the Part A variables:

```
1 wrist_rgb, wrist_depth = None, None
2 wrist_center = None
3
4 gripper_ready = False
5 gripper_closing = False
6 grab_position_started = None
7 ready_to_close = False
8 grabbed = False
9 phase = "navigate"
```

3. Add the Phase Transition

The Part A logic becomes the navigation phase. Change the beginning of the loop from:

```
1 command = {}
2 command.update(arm_stow.get_command())
```

to:

```
1 command = {}
2
3 if phase == "navigate":
4     command.update(arm_stow.get_command())
5     # Indent the remaining Part A camera and navigation logic here.
6 else:
7     # Grab logic will be added in the following steps.
8     pass
```

The common commands at the bottom of the loop remain outside the phase check:

```
1 command.update(lift_controller.get_command())
2 command = teleop.get_manual_override(command)
3 controller.set_command(command)
```

Update the first line of `navigate_to_object()` so it can change the phase:

```
1 nonlocal within_distance, phase
```

Then replace the lift update inside its flank-aligned condition with:

```
1 lift_controller.desired_state["lift_up"] = \
2     object_z - LIFT_HEIGHT_OFFSET
3
4 if lift_controller.is_at_goal():
5     phase = "grab"
6     print(f"\nPhase: {phase}")
```

For the first test, place this temporary grab block in the loop's `else` branch:

```
1 command["arm_out"] = 0.0
2 command.update(deploy_wrist.get_command())
3 command.update(gripper_open_controller.get_command())
4
5 success, wrist_rgb, wrist_depth = WRIST_CAMERA.get_frames()
6 if success:
7     wrist_center = detect_object(wrist_rgb)
8     cv2.imshow("Wrist RGB", wrist_rgb)
```

```

9     depth_display = cv2.normalize(
10         wrist_depth, None, 0, 255,
11         cv2.NORM_MINMAX, cv2.CV_8U
12     )
13     cv2.imshow("Wrist Depth", depth_display)

```

Run the program. It should navigate into the flank pose, raise the lift, switch to the wrist camera, deploy the wrist, and open the gripper.

4. Center the Object with the Wrist

Add this nested function after the navigation functions:

```

1 def center_object_with_wrist():
2     x, y = wrist_center
3     height, width = wrist_rgb.shape[:2]
4     horizontal_error = \
5         (x - width / 2 - WRIST_X_OFFSET) / (width / 2)
6     vertical_error = \
7         (y - height / 2 - WRIST_Y_OFFSET) / (height / 2)
8
9     command["wrist_yaw_counterclockwise"] = \
10         -KP_WRIST * horizontal_error
11     command["wrist_pitch_up"] = \
12         -KP_WRIST * vertical_error
13
14     cv2.circle(wrist_rgb, wrist_center, 5, (0, 255, 0), -1)
15     return horizontal_error, vertical_error

```

Call it after detecting the object in the grab branch:

```

1 if wrist_center and not grabbed:
2     horizontal_error, vertical_error = \
3         center_object_with_wrist()

```

Run the script again. The wrist pitch and yaw should keep the green point near the desired grasping location in the image.

5. Reach for the Object

Add the arm controller:

```

1 def reach_for_object(horizontal_error, vertical_error):
2     nonlocal ready_to_close
3
4     if gripper_ready:
5         distance = WRIST_CAMERA.get_depth(
6             wrist_center,
7             wrist_depth,
8             sample_radius=15,
9         )
10        if distance is not None:
11            center_error = math.hypot(
12                horizontal_error, vertical_error
13            )
14            authority = max(0.0, 1.0 - center_error)
15            command["arm_out"] = authority * KP_ARM * (
16                distance - TARGET_WRIST_DISTANCE
17            )

```

```

18         ready_to_close = (
19             center_error <= WRIST_CENTER_TOLERANCE
20             and abs(distance - TARGET_WRIST_DISTANCE)
21                 <= ARM_DISTANCE_TOLERANCE
22         )

```

At the start of the grab branch, reset the temporary condition and latch the gripper-open state:

```

1 ready_to_close = False
2 command["arm_out"] = 0.0
3 command.update(deploy_wrist.get_command())
4
5 if gripper_open_controller.is_at_goal():
6     gripper_ready = True

```

Call the reach controller immediately after wrist centering:

```

1 if wrist_center and not grabbed:
2     horizontal_error, vertical_error = \
3         center_object_with_wrist()
4     reach_for_object(horizontal_error, vertical_error)

```

Keep applying `gripper_open_controller.get_command()` for now and run the program. Once the gripper is open, the arm should extend toward 0.15m. The alignment-based authority reduces arm movement when the object is far from its desired image position.

6. Close and Lift

Add the final nested function:

```

1 def update_gripper():
2     nonlocal grab_position_started, gripper_closing, grabbed
3
4     if not gripper_closing:
5         if ready_to_close:
6             if grab_position_started is None:
7                 grab_position_started = time.monotonic()
8             elif (
9                 time.monotonic() - grab_position_started
10                 >= GRAB_POSITION_SETTLE_TIME
11             ):
12                 gripper_closing = True
13                 print("\nGrabbing position reached")
14         else:
15             grab_position_started = None
16
17     if gripper_closing:
18         command.update(gripper_close_controller.get_command())
19     else:
20         command.update(gripper_open_controller.get_command())
21
22     if (
23         gripper_closing
24         and not grabbed
25         and gripper_close_controller.is_at_goal()
26     ):
27         grabbed = True
28         lift_controller.desired_state["lift_up"] = LIFT_TOP

```

```

29     print("\nObject grabbed")
30
31     if grabbed:
32         command["arm_out"] = 0.0
33         command["wrist_pitch_up"] = 0.0
34         command["wrist_roll_counterclockwise"] = 0.0
35         command["wrist_yaw_counterclockwise"] = 0.0

```

Remove the temporary direct gripper-open command from the grab branch. Call the new function after processing the wrist frame:

```

1 update_gripper()

```

The gripper closes only after the image and depth conditions remain satisfied for half a second. Once the close controller settles, the visual servo stops and the lift target changes to its upper position.

Expected Result

Run the complete script:

```

1 uv run p4b_grab.py

```

The robot should complete this sequence:

1. Stow safely and scan for the red object.
2. Track, approach, and present its flank at approximately 0.6 m.
3. Raise the lift and deploy the wrist.
4. Center the object with wrist pitch and yaw.
5. Extend the arm until the object is approximately 0.15 m away.
6. Close the gripper and raise the object.

Keep the emergency stop accessible and clear the robot's complete motion area before every test. Collision management must remain enabled.

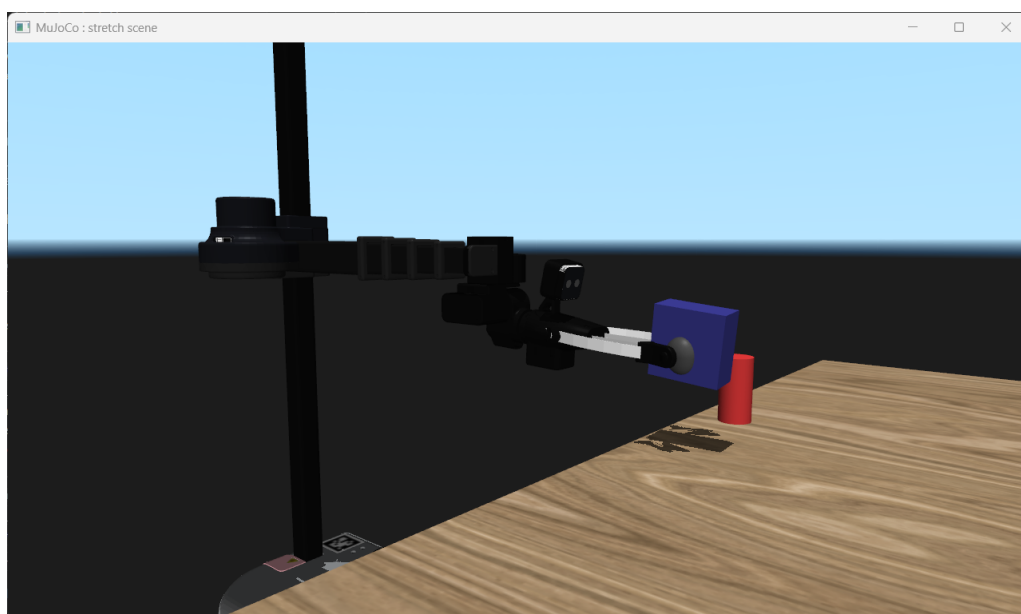


Figure 2: The completed autonomous grab with the object secured and lifted.

Deliverable

Submit a screen recording with voiceover showing `p4b_grab.py` completing the full sequence without manual assistance. The recording must show the camera windows, terminal phase messages, and the robot's complete motion from scanning through lifting the object.

Briefly explain how RGB detection, depth, `RobotTransforms`, proportional control, and the grab latches contribute to the final behavior.